

TicketMesh

Universal Configurable Ticketing & Reservation Platform — Scope Document

Executive Summary

TicketMesh is a universal, configurable, multi-provider ticketing and reservation SaaS that can be bought as software, self-hosted, or run as a full managed service with hosting. It uses a marketplace (Uber/PickMe-style) model where shops and agents connect via app, upload and sell their services, and customers also self-serve. The product is global and white-label themeable (country, currency, language, logo, colors — all by configuration, no code), delivered across a consumer Web channel and a mobile-responsive PWA app channel, plus an admin portal. It ships with 24/7 support, fraud detection, auto issue detection and auto-fix, full SDLC documentation, test and production API docs, and deploys to AWS, Kubernetes/EKS, on-premises and Docker with horizontal autoscaling.

TicketMesh — Master Requirement Register (Single Source of Truth)

- > This file is the authoritative, complete record of every requirement given for the
- > TicketMesh universal ticketing platform. It is kept in sync on every change. The final
- > .docx / .pdf scope documents are rendered from this content. It also drives the
- > reusable "platform prompt" used to build other platforms.

0. Product identity & positioning

- **Product name:** TicketMesh — Universal Configurable Ticketing & Reservation Platform.
- Marketed as a global, production-standard SaaS that can be: (a) bought as software, (b) self-hosted on the buyer's own server, or (c) run as a full managed service (hosting included) by us.
- Target deploy targets: AWS, Kubernetes (EKS), on-premises, Docker, single server — with horizontal autoscaling and easy deployment everywhere.
- Prefer free / standard production SaaS-level tooling and self-hostable components (Docker, PostgreSQL, Redis, Kafka, DuckDB, Prometheus, Grafana, local AI models).

1. Core scope (from master blueprint — all 53+ parts)

Universal, configurable, multi-provider ticketing & reservation covering **bus, train, movie, events, sports, flight, ferry, attractions** and future domains. Key pillars:

1. Universal ticket model: TicketProduct, TicketType, Provider, Inventory, Offer, Reservation, Booking, Ticket, Policy.
2. Multi-domain support (bus seats, train berths/classes, movie shows, event/sports seats).
3. Multi-provider aggregation: parallel search, normalize, dedupe, compare, rank.
4. Provider adapter framework: search, availability, seat map, price, hold, release, book, confirm, get booking, cancel, refund, modify, ticket retrieval, validation.
5. Provider onboarding + capability system (providers declare supported capabilities).
6. Configuration-driven product engine (no-code/low-code) — workflows, fields, seats, policies, fees, taxes, promotions, notifications, validation, all configurable.
7. Universal search; AI conversational search; multi-ticket/multi-service Trip; smart trip planner.
8. Inventory management (AVAILABLE -> HELD -> CONFIRMED; HELD -> EXPIRED -> AVAILABLE).
9. Reservation & concurrency (idempotency keys, optimistic locking, expiry, reconciliation).
10. Booking engine, payment abstraction (cards/wallets/bank/refunds/webhooks), refund & cancellation, ticket issuing & validation (QR/barcode), pricing & fare engine, promotion engine, customer & loyalty.
11. Admin & operations console; no-code/low-code configuration; multi-tenancy SaaS.
12. AI assistant; AI agent architecture (controlled tools, least privilege, financial confirmation, audit); recommendation engine; seat recommendation; price prediction;

dynamic pricing; fraud & cyber risk; disruption/recovery intelligence.

13. Ticket intelligence / data analytics; AI data analyst (read-only semantic layer).

14. Data platform (Kafka -> analytics; Postgres; Parquet/DuckDB offline; S3).

15. ML platform (demand forecasting, cancellation prediction, fraud scoring, recommender, anomaly detection). Event-driven architecture. Microservices + API gateway/edge.

16. Security architecture (OAuth2/OIDC, RBAC/ABAC, tenant isolation, encryption, secrets, audit, rate limiting, mTLS, PII minimization, zero-trust).

17. AI security (prompt-injection defense, PII redaction, tool authorization, output validation, retrieval filtering, DLP, model allowlists, audit, adversarial eval).

18. DevOps/DevSecOps, CI/CD, Kubernetes & GitOps (EKS, Helm, Argo CD/Rollouts, KEDA, HPA), AWS architecture, free/offline dev strategy.

19. Frontends: React + TypeScript web (consumer + admin); optional React Native mobile later. Testing & quality, observability, monorepo layout, delivery roadmap, MVP + DoD.

2. Additional requirements given during the build session (all MUST be in docs)

2.1 Marketplace / agent model (Uber / PickMe style)

- ****Customer / Agent / Admin**** users must all exist.
- Any ****shop / agent**** can connect its service to the platform and give/sell to customers.
- A customer can also use the platform directly by themselves (self-serve).
- Agents/shops connect ****via app****, upload their ****ticket details and services****, sell tickets to customers.
- Shop owner can ****build/manage their shop using just their app****; admin manages overall and can ****monitor**** everything.

2.2 Global / white-label / themeable (WordPress / Uber / PickMe style)

- Standard ****global product usable from any country, any currency, any language****.
- ****Everything configurable**** like WordPress themes or Uber/PickMe: theme colors, images, ****logo****, and anything else the shop needs to change — purely via configuration (no code).
- Multi-tenant SaaS white-label branding.

2.3 Channels

- ****Web**** channel (consumer web) and ****app**** channel (mobile-responsive PWA installable).
- Admin portal for overall management + monitoring.

2.4 Deployment & operations (production SaaS)

- Deployable to ****AWS, Kubernetes, on-prem, Docker**** — horizontal ****autoscaling****, easy deploy.
- Buy-as-software, self-host, or ****full managed service with hosting****.
- Mostly ****free + standard**** production SaaS-level tooling.

2.5 Testing / API tooling

- Use ****WireMock**** for offline provider/API stubbing where anything cannot connect

immediately.

- Provide **test** API documentation and **production** API documentation.

2.6 Support & 24/7 operations

- **Support service** and a **24/7 support portal with management**.
- Consider and include **all e-services needed for a production application** — build them all.
- Development **release notes**, **QA release notes**, and any document needed **start-to-end** — prepare them all: architecture diagrams, manuals, **SDLC process**, as a **standard company**.

2.7 Automation / proactive capabilities

- **Auto-detect issues** (live + dev + QA issues).
- **Fraud detection system**.
- **Auto-fix** live issues, dev issues and QA issues.
- **Full automation with a support system**.
- Monitoring, health checks, observability.

3. Product-name & business direction guidance (user asked for suggestion)

- Blueprint name: **TicketMesh**.
- Recommended platform brand: **TicketMesh** (keeps blueprint continuity, professional, sellable). Tagline: "All your tickets, one platform."
- The user additionally wants a **multi-product company/portfolio** later: e-commerce, e-hotel management, e-channel, fintech/lending platforms, daycare platform, learning platform, elder-care/child-care, company sites, web portals, apps, portfolios — all connected to WhatsApp/LinkedIn/Facebook (multi-channel). Ticketing is built first; the same reusable **platform prompt + architecture + docs** will be reused for each.
- Sell through multiple places (like WordPress marketplace model): SaaS marketplace, theme marketplace, managed-service offering, on-prem licensing.

3.5 Delivery decision & blueprint coverage

- **Delivery decision:** the blueprint's multi-microservice topology is delivered this build as a **pragmatic modular single-backend** (ticketmesh-core Spring Boot) with clear within-core module boundaries. This keeps the build coherent and shippable while remaining microservice-ready (splitting later is a boundary refactor, not a rewrite).
- **Coverage mapping:** see docs/blueprint-coverage.md for the honest per-part status of the 53+ part master blueprint (BUILT / PARTIAL / DOC). It lists which parts are fully implemented + tested, which are represented by the modular/config/adaptor surface, and which are documented as planned.

4. Non-negotiable (from CLAUDE.md — authorship/quality)

- Author: **sangiya** (asan935para@gmail.com). No "Claude"/"AI-generated" anywhere.
- Production-grade only; real tests with real assertions; professional repo names.

- Data safety: never commit real production data, credentials, or customer info.
- No employer proprietary content.

Author

Prepared by sangiya — Associate Software Architect.